

Reviewer Pack — Minimal Rank of Twisted K-Theory vs Communication Complexity...

Entry 76a317ada648 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 08:05:16 UTC

Minimal Rank of Twisted K-Theory vs Communication Complexity of XOR-AND Trees

Entry ID: 76a317ada648 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 08:05:16 UTC

1. Conjecture

Field A (mathematical branch): Twisted K-Theory **Field B** (complexity object): Communication Complexity (XOR-AND Trees)

Statement:

{‘s1’: ‘For any k -CNF formula F with n variables, the minimal rank of the twisted K-theory module associated with F is at most $O(n^{1/2+\epsilon})$ for some $\epsilon > 0$.’, ‘s2’: ‘The communication complexity of evaluating XOR-AND trees associated with F is at least $\Omega(\min(n, \log(1/\delta)))$, where δ is the probability that a random variable x from the input space satisfies $F(x)$.’, ‘s3’: ‘Moreover, for any $\epsilon > 0$, there exists a polynomial-time computable mapping from k -CNF formulas to twisted K-theory modules such that the image of an instance with communication complexity greater than $\Omega(\min(n, \log(1/\delta)))$ has twisted K-theory rank at least $n^{1/2+\epsilon}$.’}

Rationale (proposer’s reasoning):

{‘s1’: ‘Twisted K-Theory provides a framework for studying topological properties that are invariant under certain kinds of transformations, which could potentially reveal new insights into the structure of Boolean functions.’, ‘s2’: ‘The communication complexity of XOR-AND trees captures the essential difficulty of computing Boolean functions, and a connection with twisted K-theory might expose underlying geometric or algebraic structures in complexity theory.’, ‘s3’: ‘A constructive mapping from k -CNF formulas to twisted K-theory modules would enable direct computation of the invariants and verification of the conjecture.’}

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 03c94dd2853d3b57

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a k -CNF formula F with n variables, the minimal rank of its twisted K-theory module is at most $O(n^{1/2+\varepsilon})$ and the communication complexity of XOR-AND trees associated with F satisfies $\Omega(\min(n, \log(1/\delta)))$ for some $\varepsilon > 0$ and δ .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "twisted K-theory" AND "communication complexity" AND "XOR-AND trees"
- "minimal rank" Twisted K-theory "communication complexity" XOR-AND - "polynomial-time computable mapping" k -CNF formulas twisted K-theory module communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        # Find pivot row
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = A[i][i]
        for j in range(n):
            A[i][j] /= factor

        for j in range(i+1, m):
            factor = A[j][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

    return A
```

```

def rank(A):
    rref = gaussian_elimination(A)
    rank = 0
    for row in rref:
        if any(row):
            rank += 1
    return rank

def generate_kcnf(n, k):
    variables = list(range(1, n+1))
    clauses = []
    for _ in range(k):
        clause = random.sample(variables, random.randint(1, n))
        clause.append(-random.choice(clause)) # Ensure at least one negated variable
        clauses.append(clause)
    return clauses

def communication_complexity(F):
    n = len(F[0])
    delta = sum(1 for clause in F if all(x in clause or -x in clause for x in range(1, n+1))) / (2**n)
    return max(n, math.log(1/delta))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        F = generate_kcnf(n, n)
        A = [[0] * (n+1) for _ in range(n+1)]

        # Construct the twisted K-theory matrix
        for clause in F:
            for x in clause:
                if x > 0:
                    A[x][x] += 1
                else:
                    A[-x][-x] -= 1

        tk_rank = rank(A)
        cc = communication_complexity(F)

        results.append({
            "n": n,
            "tk_rank": tk_rank,
            "cc": cc,
            "conjecture_holds": tk_rank <= n**(0.5 + 1e-6) and cc >= min(n, math.log(1/0.01))
        })

    metric_value = sum(result["tk_rank"] for result in results)
    instances_tested = len(results)
    conjecture_holds = all(result["conjecture_holds"] for result in results)
    counterexample = "" if conjecture_holds else "Counterexample found"

    return {
        "metric_name": "twisted_k_theory_rank",

```

```

        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"First failing seed\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c8f3e22b.py", line 107, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c8f3e22b.py", line 78, in run_trial
    tk_rank = rank(A)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c8f3e22b.py", line 40, in rank
    rref = gaussian_elimination(A)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c8f3e22b.py", line 31, in gaussian_elimination
    A[i][j] /= factor
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture’s claims. | next: Re-run the test with appropriate error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13594
2	preregistration	ollama_remote	glm4:latest	0	0	6989
3	novelty	ollama_remote	glm4:latest	0	0	4832
4	novelty	ollama_remote	glm4:latest	0	0	6206
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42839
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8854
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11386
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12168
9	judge	ollama_remote	glm4:latest	0	0	10991

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117858 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/76a317ada648.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/76a317ada648.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/76a317ada648.tar.gz (if generated)